Timestamp Event Service Design

1. Problem

The service receives a customer event and stores when it happened in a database. The main requirements are simple: validate the request, write the event safely, and return the stored record.

At a high level: a client sends a customer ID, the app checks the input, PostgreSQL records the event and timestamp, and the API returns the saved row after the write succeeds.

2. Implemented local design

This repository implements a local PostgreSQL design:

- one Flask application
- one PostgreSQL database running locally
- Gunicorn for local process management
- environment-variable configuration for database connection settings
- validation of request payloads
- committed inserts with a returned saved record
- `GET /health` for database readiness and `GET /health/live` for application liveness

The table keeps `event_id` as the primary key and does not enforce uniqueness on `customer_id`. Multiple events per customer are allowed.

3. Local request flow

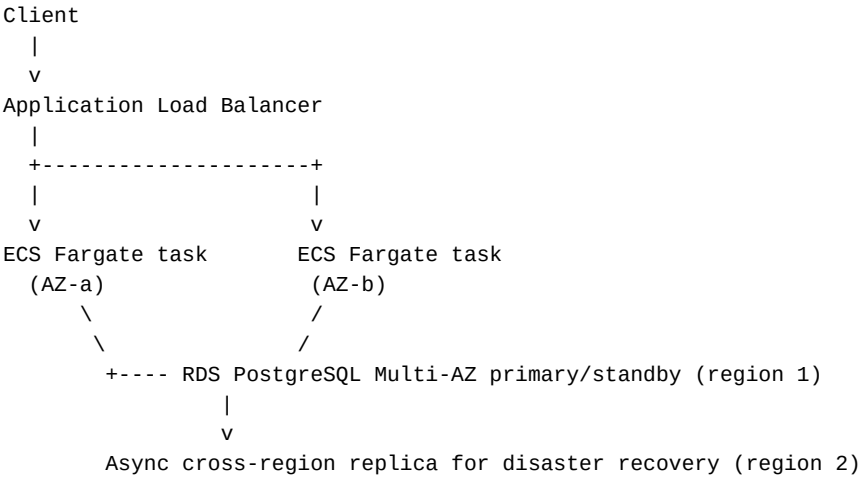
1. Client sends a POST request with `customer_id`.
2. The Flask app validates the JSON body and customer ID.
3. The app inserts a row into PostgreSQL.
4. PostgreSQL uses `DEFAULT NOW()` for the timestamp.
5. PostgreSQL records the transaction start time, not the original customer action time.
6. The app returns the saved record after commit.
7. The database connection is closed after the request finishes.

4. Why this design works

- The write path is simple.
- The application is stateless, so multiple instances can point at the same database.
- PostgreSQL provides a durable single source of truth.

5. Proposed production architecture

This is the production design I would propose for a small stateless service on AWS:



This is a proposed architecture, not the local implementation.

6. Availability and redundancy

- Run application tasks across at least two availability zones.
- Put the app behind an Application Load Balancer.
- Keep the database primary in one AZ and a standby in another AZ within the same region.
- Use synchronous replication inside a region for the primary/standby pair.
- A restart or a zone failure should redirect traffic to healthy app instances and then recover the database.
- Recovery time and possible data loss should be described explicitly rather than promised as seamless failover.

7. Elasticity

This service can scale by increasing or decreasing the number of app tasks based on demand.

A starting policy could be:

- minimum 2 tasks
- maximum 10 tasks
- CPU target 60%

These values are initial assumptions requiring load testing and real traffic measurements. They are not implemented locally.

Important points:

- App scaling is limited by database capacity and connection limits.
- Read replicas do not increase insert/write throughput.
- A connection pool or proxy would be a future improvement for high concurrency.
- Gunicorn and local PostgreSQL do not implement demand-based autoscaling.

8. Multi-DC and disaster recovery

A second region should hold an asynchronous PostgreSQL replica for disaster recovery. This is a proposed design only.

- synchronous replication protects a region from local failures
- asynchronous cross-region replication helps protect against regional failure

- regional recovery requires isolating the old writer, promoting the replica, updating application connections, and shifting traffic
- asynchronous replication can lose recent acknowledged writes
- example targets for discussion are RTO 30 minutes and RPO 5 minutes, clearly labelled as proposed targets and not validated production numbers

9. Automation gaps and proposed follow-up work

What is automated locally:

- PostgreSQL must be installed and running separately
- Gunicorn starts the Flask service locally
- the app validates the input and writes to PostgreSQL
- the app uses environment variables for database configuration

What is not implemented here:

- cloud deployment
- infrastructure as code
- CI/CD automation
- autoscaling policies
- database migration automation
- cross-region failover automation
- monitoring, alerting, and backup restore drills
- TLS and secrets management

A follow-up plan would include:

- Terraform or similar IaC for AWS resources
- ECS Fargate or a managed app platform
- automated database migration jobs
- monitoring and alerting
- backup and point-in-time recovery validation

10. Why Flask, PostgreSQL, and a synchronous write path

I chose Flask because it keeps the implementation small and easy to explain. I chose PostgreSQL because it is a reliable relational database that supports transactions, durability, and straightforward data modelling. I kept the write path synchronous because the assignment requires a committed event record and a clear success/failure outcome. For a small service, this is simpler to reason about than introducing queues or asynchronous processing before the data is durable.

11. Production follow-ups

- authentication and TLS
- least-privilege database access
- secrets management
- structured logs and operational monitoring
- alerting
- idempotency keys for retry protection
- backup and restore drills
- queueing is optional and changes the meaning of acceptance versus confirmed persistence